

SP-201 Red Multi-Agent Research & Fact-Checking Pipeline

SOFTWARE DESIGN DOCUMENT(SDD)

Senior Project Section 2
Spring 2026
February 15, 2026
Professor Perry



Melike Ozcelik
Team Leader
Developer



Raaid Khan
Developer
Documentation



Nha Phan
Developer



Afra Kurudirek
Documentation

1.	INTRODUCTION AND OVERVIEW	4
1.1.	PURPOSE OF THE SDD	4
2.	DESIGN CONSIDERATIONS	5
2.1.	ASSUMPTIONS AND DEPENDENCIES	5
2.2.	GENERAL CONSTRAINTS	5
2.3.	GOALS AND GUIDELINES	6
2.4.	DEVELOPMENT METHODS	6
3.	ARCHITECTURAL STRATEGIES	6
4.	SYSTEM ARCHITECTURE	7
4.1.	SUBSYSTEM ARCHITECTURE	8
5.	DETAILED SYSTEM DESIGN	8
5.1.	MANAGER AGENT	9
5.1.1.	<i>Classification</i>	9
5.1.2.	<i>Definition</i>	9
5.1.3.	<i>Responsibilities</i>	9
5.1.4.	<i>Constraints</i>	10
5.1.5.	<i>Composition</i>	10
5.1.6.	<i>Uses/Interactions</i>	10

	5.1.7.....	Resources
	10	
	5.1.8.....	Processing
	10	
	5.1.9.....	Interface/Exports
	11	
11	5.2.....	SEARCH AGENT
	5.2.1.....	Classification
	11	
	5.2.2.....	Definition
	11	
	5.2.3.....	Responsibilities
	11	
	5.2.4.....	Constraints
	11	
	5.2.5.....	Composition
	11	
	5.2.6.....	Uses/Interactions
	12	
	5.2.7.....	Resources
	12	
	5.2.8.....	Processing
	12	
	5.2.9.....	Interface/Exports
	12	
12	5.3.....	SYNTHESIZER AGENT
	5.3.1.....	Classification
	12	
	5.3.2.....	Definition
	12	
	5.3.3.....	Responsibilities
	12	
	5.3.4.....	Constraints
	12	
	5.3.5.....	Composition
	13	
	5.3.6.....	Uses/Interactions
	13	
	5.3.7.....	Resources
	13	
	5.3.8.....	Processing
	13	
	5.3.9.....	Interface/Exports
	13	
13	5.4.....	FACT CHECKER AGENT
	5.4.1.....	Classification
	13	
	5.4.2.....	Definition
	13	
	5.4.3.....	Responsibilities
	13	
	5.4.4.....	Constraints
	14	

5.4.5.....	Composition
14	
5.4.6.....	Uses/Interactions
14	
5.4.7.....	Resources
14	
5.4.8.....	Processing
14	
5.4.9.....	Interface/Exports
14	
5.5.....	SESSION STATE MANAGER
14	
5.5.1.....	Classification
14	
5.5.2.....	Definition
14	
5.5.3.....	Responsibilities
15	
5.5.4.....	Constraints
15	
5.5.5.....	Composition
15	
5.5.6.....	Uses/Interactions
15	
5.5.7.....	Resources
15	
5.5.8.....	Processing
15	
5.5.9.....	Interface/Exports
15	
6.....	GLOSSARY
15	
7.....	BIBLIOGRAPHY
16	

1. Introduction and Overview

1.1. Purpose of the SDD

This System Design Document (SDD) aims to provide an architectural and technical description of the SP-201 Red Multi-Agent Research & Fact-Checking Pipeline. The Software Requirements Specification (SRS)'s functional and non-functional requirements are translated into a structured system architecture and component-level

design in this document.

This SDD offers recommendations for system integration, testing, deployment, and future expansion. Developers, reviewers, and other stakeholders engaged in the design and implementation process might use it as a reference.

2. Design Considerations

2.1. Assumptions and Dependencies

The following presumptions and dependencies form the foundation of this system's design:

- Python 3.10 or later will be used to develop the system.
- Agent orchestration will be done using Google ADK.
- The major language model provider will be Gemini API.
- For external data retrieval, a public search API (like DuckDuckGo) will be accessible.
- English-language prompts will be given by users.
- The system will be developed in a single-user setting.
- API communication requires an internet connection.
- Other agents (such bias detection or citation formatting agents) might be added in future extensions.

2.2. General Constraints

System design is greatly impacted by the following limitations:

- Limits on Tokens and Context Windows: The system must control agent answers and prompt size while adhering to LLM token restrictions.
- Rate Limits for External APIs: Response delays and rate limits may be enforced by external APIs.
- Performance Restrictions: Multi-agent execution must finish in a reasonable amount of time (usually less than a minute for ordinary prompts).
- Security Requirements: Environment variables must be used to safely store API keys.
- Network Communication: HTTPS is used for all external communications.

- Testing Requirements: For validation and debugging, the system must have repeatable execution processes.
- Architectural choices like state management, retry methods, and modular breakdown are influenced by these limitations.

2.3. Goals and Guidelines

The following are the main objectives driving the design:

- Agents' responsibilities are divided into modules.
- Reasoning transparency and citation traceability
- decrease in hallucinogenic outputs
- Transitions between workflow states are clear.
- Flexibility for upcoming agents
- Clean module separation for maintainability
- Correctness and verification are given precedence over raw performance speed by the system.

2.4. Development Methods

The design process for the system is progressive and modular. Before being incorporated into the entire workflow, every agent component is independently developed and evaluated.

Agents are organized as discrete modules with well-defined interfaces using an object-oriented methodology. Verification logic, agent coordination, and prompt engineering are all enhanced through iterative refining.

3. Architectural Strategies

The architecture pattern used by the system is Manager–Worker.

Strategy 1: Orchestration in Hierarchy

Workflow progression is managed by a central Manager Agent. Search, Synthesizer, and Fact-Checker are examples of worker agents that function independently but are coordinated by structured message forwarding.

A monolithic single-agent design was taken into consideration as an alternative.

Reason Rejected: Poor modularity, less control over verification, and decreased transparency.

Strategy 2: Model of Sequential Execution

Agents work in regulated, sequential stages:

Pending → Dissecting → Looking → Combining → Verifying → Completing

Parallel uncontrolled execution was taken into consideration.

Increased synchronization difficulty and inconsistent state danger were the reasons for rejection.

Strategy 3: Components of Modular Agents

Every agent has distinct input/output schemas and is implemented as a stand-alone software module.

This makes it possible for:

- Self-directed testing
- Future agent replacement
- Scalability to accommodate more agents

4. System Architecture

The system is broken down into the high-level subsystems listed below:

1. Subsystem for User Interface
2. Subsystem for Manager Agents
3. Subsystem for Search Agents
4. Subsystem for Synthesizer Agents
5. Subsystem of Fact-Checker Agents
6. Subsystem for Session State Management
7. Subsystem for External API Integration

The UI gathers prompts and shows the outcomes.

The execution flow is coordinated by the Manager Agent.

Structured external data is retrieved by the search agent.

A well-organized research draft is created by the Synthesizer Agent. The Fact-Checker Agent verifies statements by comparing them to the original sources. The Session State Manager preserves execution context and keeps track of workflow transitions. Structured data items are used for communication between all components.

4.1. Subsystem Architecture

User Interface:

- Prompt input field
- Status Display
- Markdown output display with citations

Agent Coordination Flow:

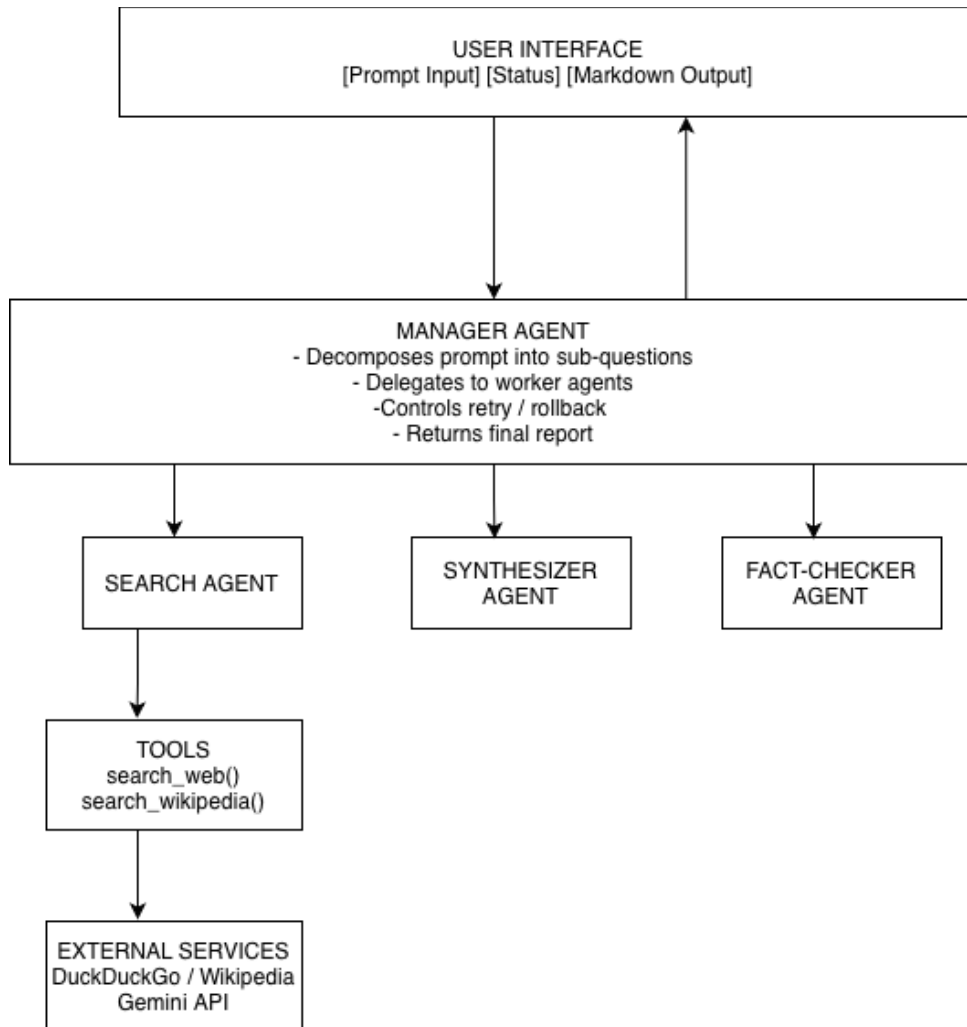
User

→ Manager Agent
→ Search Agent
→ Manager Agent
→ Synthesizer Agent
→ Fact Checker Agent
→ Manager Agent
→ User

If validation fails, control returns to the Search or Synthesizer stage.

5. Detailed System Design

The main software components mentioned in the System Architecture section are described in detail in this section. Every component is explained in terms of its description, classification, roles, limitations, composition, interactions, resources, processing behavior, interface, and exports.



5.1. Manager Agent

5.1.1. Classification

Application level orchestration module responsible for system workflow control.

5.1.2. Definition

The multi-agent research workflow is carried out under the direction of the Manager Agent. The user query is interpreted, broken down into structured sub-questions, assigned to worker agents, state transitions are handled, and the user receives the final validated report. The orchestration specifications given in the Software Requirements Specification are directly satisfied by this component.

5.1.3. Responsibilities

- Accept user research prompt
- Decompose prompt into at least three structured sub-questions

- Delegate sub-questions to Search Agent instances
- Initiate synthesis and validation stages
- Handle workflow rollback if validation fails
- Track system state transitions
- Aggregate and return final output

5.1.4. Constraints

- Must operate within LLM token and context window limits
- Must maintain valid workflow state transitions
- Must handle API failures without corruption session state
- Must prevent infinite validation loops

Precondition: A valid prompt string must be provided

Postcondition: A validated research report or structured error response is returned

5.1.5. Composition

The Manager Agent composes and coordinates:

- Search Agent
- Synthesizer Agent
- Fact Checker Agent
- Session State Manager

5.1.6. Uses/Interactions

- Calls Search Agent for data retrieval
- Passes structured results to Synthesizer Agent
- Sends synthesized draft to Fact Checker Agent
- Receives validation status and determines next state

5.1.7. Resources

- LLM API (Gemini)
- Network Connectivity
- System memory for session state
- External search APIs

Race conditions are avoided through controlled sequential execution. No concurrent agent execution is implemented.

5.1.8. Processing

Implements a deterministic state driven workflow:

Pending →Decomposing →Searching →Synthesizing →Fact Checkng →Finalizing

If validations fails, control transitions back to Searching or Synthesizing Error handling includes retry logic and controlled rollback.

5.1.9. Interface/Exports

process_prompt(prompt:string) → final_report
get_workflow_status() → state_identifies

5.2. Search Agent

5.2.1. Classification

External data retrieval module.

5.2.2. Definition

The Manager Agent submits sub-questions to the Search Agent, which then collects pertinent data from open search APIs. For further processing, it organizes the data that has been retrieved into citation-preserving objects.

5.2.3. Responsibilities

- Submit search queries to approved APIs
- Retrieve relevant textual content
- Extract source titles and URLs
- Return structured search result objects

5.2.4. Constraints

- Subject to API rate limits
- Dependent on network availability
- Limited by external API schema structure
- Must return results in structured format

Precondition: Valid query string provided

Postcondition: Structured search result object returned.

5.2.5. Composition

- API wrapper module
- Response parser
- Citation extractor

5.2.6. Uses/Interactions

- Invoked by Manager Agent
- Returns structured results for synthesis

5.2.7. Resources

- External search API
- Network connectivity
- Temporary memory for parsing results

5.2.8. Processing

- Submit query
- Receive API response
- Parse JSON response
- Extract content and metadata
- Structure results

Includes retry logic and error handling.

5.2.9. Interface/Exports

search(query:string) → structured_results

5.3. *Synthesizer Agent*

5.3.1. Classification

Content generation module.

5.3.2. Definition

While maintaining citation mappings, the Synthesizer Agent incorporates structured search results into a logically ordered, cogent research draft.

5.3.3. Responsibilities

- Organize retrieved data by topic
- Generate structured paragraphs
- Preserve citation references
- Format output in markdown

5.3.4. Constraints

- Must operate within LLM token limits
- Must not fabricate unsupported claims

- Must preserve citation mapping

Precondition: Structured search results available.

Postcondition: Draft report returned.

5.3.5. Composition

- Prompt construction logic
- LLM response handler
- Markdown formatter

5.3.6. Uses/Interactions

- Receives data from Manager Agent
- Sends draft to Fact Checker Agent

5.3.7. Resources

- LLM API
- System memory for draft storage

5.3.8. Processing

1. Construct structured synthesis prompt
2. Submit to LLM
3. Format output
4. Return draft

5.3.9. Interface/Exports

`synthesize(search_results) → draft_report`

5.4. *Fact Checker Agent*

5.4.1. Classification

Validation and verification module.

5.4.2. Definition

By comparing assertions with the retrieved source information, the Fact-Checker Agent assesses synthesized content and looks for contradictions or unjustified claims.

5.4.3. Responsibilities

- Compare claims to source content
- Identify unsupported statements
- Return validation status

- Trigger reprocessing if necessary

5.4.4. Constraints

- Limited to retrieved data
- Cannot validate beyond provided sources
- Dependent on LLM evaluation accuracy

Precondition: Draft and search results available.

Postcondition: Validation result returned

5.4.5. Composition

- Claim extraction logic
- Source comparison mechanism
- Validation decision logic

5.4.6. Uses/Interactions

- Receives draft from Manager
- Returns validation result to Manager

5.4.7. Resources

- LLM API
- Structured search results

5.4.8. Processing

1. Extract claims
2. Cross reference against sources
3. Identify inconsistencies
4. Return validation status

5.4.9. Interface/Exports

validate(draft, sources) → validation_status

5.5. Session State Manager

5.5.1. Classification

State tracking and workflow control module

5.5.2. Definition

Maintains workflow state transitions and session context across agent interactions.

5.5.3. Responsibilities

- Track execution stage
- Store intermediate outputs
- Provide traceability

5.5.4. Constraints

- Must maintain consistent state
- Must prevent invalid transitions

5.5.5. Composition

- State enum definitions
- Session storage object

5.5.6. Uses/Interactions

- Accessed by Manager Agent
- Logs state transitions

5.5.7. Resources

- System Memory
- Optional database storage

5.5.8. Processing

Update state based on workflow stage transitions

5.5.9. Interface/Exports

`update_state(new_state)`
`get_state()`

6. Glossary

Agent – Autonomous software component performing a defined task

Manager – Worker Pattern – Hierarchical orchestration design pattern

LLM – Large Language Model

Hallucination – Unsupported context across agent stages

Session State – Stored execution context across agent stages

7. Bibliography

Google ADK Documentation
Google AI Studio Documentation
Multi-Agent Systems Design Literature
Software Engineering Design Pattern References